

Rapport Technique

HealthAI Coach — Projet Data Engineering
Simplon Formation Data Engineer

Alexandre Laugier* Florian Abgrall† Johane Decamps William Mibelli‡

Juillet 2026

*Rôle A — Architecte / Lead Data Engineer : schéma base de données, interface admin (Gradio), dashboard Metabase, dockerisation

†Rôle B — API REST (FastAPI)

‡Rôle C — ETL (extract/transform/load), orchestration Airflow

Table des matières

1	Introduction	3
2	Architecture du système	3
2.1	Vue d'ensemble	3
2.2	Choix technologiques	3
3	Modélisation des données	4
3.1	Modèle Conceptuel de Données — périmètre OLTP	4
3.2	Contraintes d'intégrité et conventions	4
4	Pipeline ETL	5
4.1	Structure du pipeline	5
4.2	Données sources	6
4.3	Qualité des données	6
5	API REST	7
5.1	Structure	7
5.2	Endpoints principaux	7
5.3	Abonnements et microservices IA	7
6	Interface admin (Gradio) et dashboard Metabase	8
6.1	Interface admin	8
6.2	Accessibilité RGAA	8
6.3	Dashboard Metabase	9
7	Dockerisation	10
8	Tests et CI/CD	11
8.1	Suite de tests	11
8.2	Pipeline CI	11
9	Bilan et obstacles	12
9.1	Obstacles techniques rencontrés	12
9.2	Points positifs	12
10	Conclusion	12
A	Extrait du schéma SQL — tables principales	13

1 Introduction

HealthAI Coach est une plateforme de coaching santé : suivi nutritionnel, suivi de l'activité physique, relevés biométriques et profil médical déclaratif d'un utilisateur, avec génération de recommandations personnalisées (diététiques, puis plans d'entraînement/nutrition via un microservice IA à venir). Ce dépôt couvre exclusivement le **backend data** du projet : modélisation et administration de la base de données, pipeline ETL d'ingestion depuis des sources hétérogènes (Kaggle, API publique d'exercices), API REST, supervision de la qualité des données, et conteneurisation de l'ensemble — pas le frontend utilisateur final, hors périmètre de cette formation.

Le modèle économique conditionne plusieurs choix techniques documentés dans ce rapport : offre free-mium (**free/premium/premium_plus**) avec fonctionnalités IA réservées aux paliers payants, et offre B2B en marque blanche pour des organisations partenaires (salles de sport, mutuelles, entreprises), provisionnée par un administrateur plutôt qu'en self-service.

Travail réparti en trois rôles : Rôle A — architecture, base de données, interface admin (qualité des données), dashboard Metabase et dockerisation (Alexandre Laugier) ; Rôle B — API REST FastAPI (Florian Abgrall) ; Rôle C — pipeline ETL et orchestration Airflow (Johane Decamps, William Mibelli). Le développement suit un flux Pull Request systématique (une étape = une PR, tests obligatoires, revue avant fusion) détaillé dans `docs/plan_de_developpement.md`.

2 Architecture du système

2.1 Vue d'ensemble

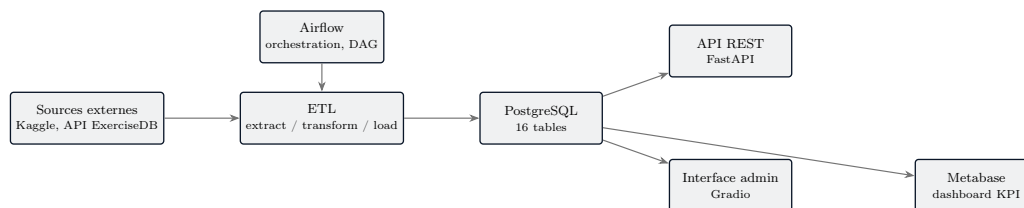


FIGURE 1 – Vue d'ensemble — l'ETL alimente PostgreSQL (orchestré par Airflow), consommé en aval par l'API REST, l'interface admin et Metabase. Tout le périmètre (hors `api/airflow`, en attente des Dockerfile des équipes concernées à date de rédaction) est conteneurisé via `docker compose` (cf. §7).

2.2 Choix technologiques

Composant	Technologie
Base de données	PostgreSQL
Migrations	Alembic
Orchestration	Apache Airflow
API	FastAPI (SQLAlchemy, Pydantic, JWT)
Dashboard	Metabase
Interface admin	Gradio
Conteneurisation	Docker / Docker Compose
CI	GitHub Actions

TABLE 1 – Stack technique (`README.md`).

Deux choix méritent d'être justifiés au-delà du tableau : **PostgreSQL** plutôt qu'un SGBD NoSQL, parce que le domaine (utilisateurs, journaux nutrition/exercice historisés, contraintes d'intégrité inter-tables) est structurellement relationnel — les nombreuses contraintes `CHECK/FOREIGN KEY` qui portent une partie significative des règles métier (cf. §3.2) auraient dû être réimplémentées côté application dans un magasin sans schéma. **Gradio** plutôt qu'un framework front complet (React, Streamlit) pour l'interface admin, parce qu'il s'agit d'un outil interne à faible surface (un seul écran, quelques formulaires) où la rapidité de mise en œuvre prime sur la personnalisation visuelle — au prix de limitations d'accessibilité propres au composant `Dataframe`, documentées en §6.

3 Modélisation des données

Le schéma relationnel (`database/schema/ddl_postgres.sql`) compte 16 tables, réparties en cinq domaines fonctionnels autour de l'entité centrale **USERS** : nutrition, activité physique, suivi déclaratif de santé, abonnements, et contenu généré par IA. Chaque table porte des commentaires SQL (`COMMENT ON TABLE/COLUMN`) documentant directement en base la signification de chaque champ — en particulier les conventions non triviales comme les clés d'idempotence `UNIQUE(source, external_id)`.

3.1 Modèle Conceptuel de Données — périmètre OLTP

La figure 2 présente le cœur du modèle : nutrition, activité physique, suivi de santé déclaratif et contenu généré par IA (`workout_plans/nutrition_plans`), tous rattachés à **USERS**. La figure 3 présente le domaine abonnements, ajouté dans un second temps pour l'offre B2C/B2B.

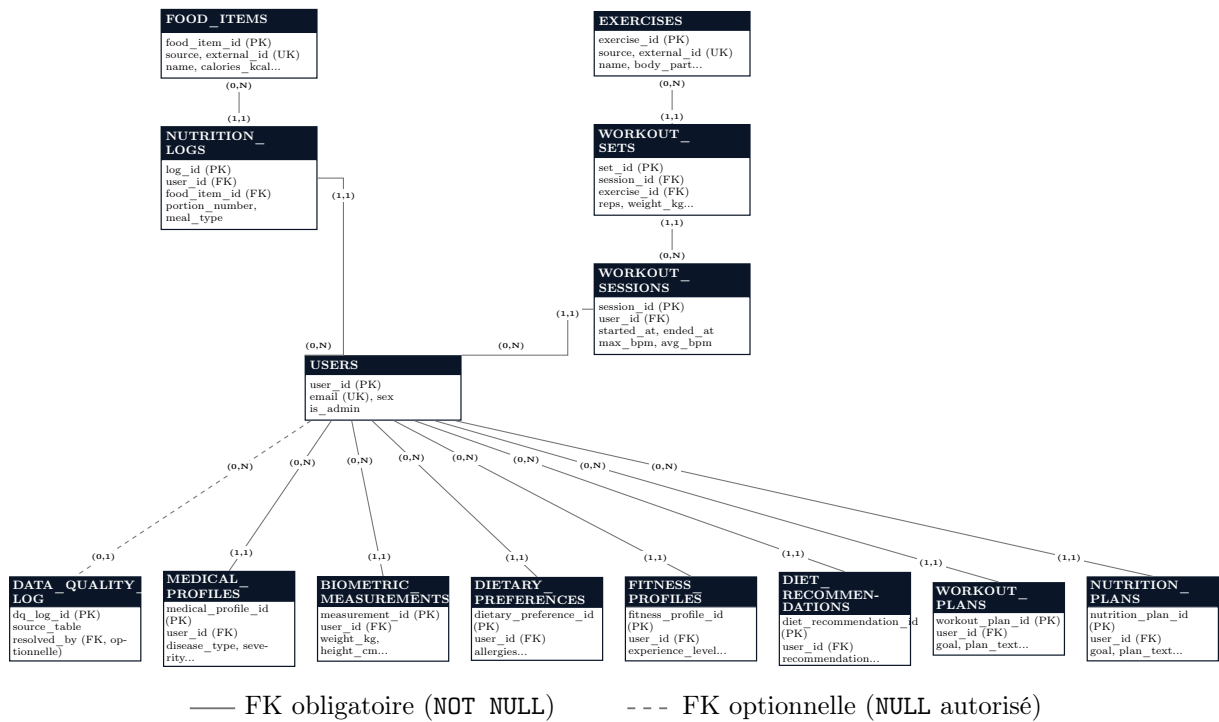


FIGURE 2 – MCD — cœur du modèle (nutrition, activité physique, suivi de santé, contenu IA). Cardinalités au format Merise (min,max), lues côté entité adjacente.

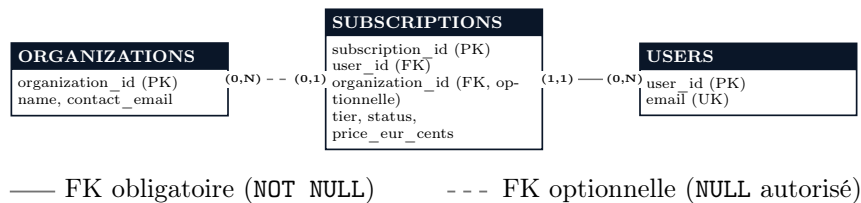


FIGURE 3 – MCD — domaine abonnements. `organization_id` n'est renseigné que pour `tier = 'b2b'` (contrainte `chk_subscriptions_b2b_organization`).

3.2 Contraintes d'intégrité et conventions

Le schéma s'appuie sur PostgreSQL pour porter un maximum de règles métier directement en base plutôt que dans le code applicatif, avec quatre familles de contraintes récurrentes :

- **Domaines de valeurs** — `CHECK` sur les colonnes à choix fermé : `sex` IN ('F', 'M', 'other'), `severity` IN ('info', 'warning', 'error', 'critical') (`data_quality_log`), `meal_type` IN ('breakfast', 'lunch', 'dinner', 'snack'), `tier` IN ('free', 'premium',

'premium_plus', 'b2b'). Ces contraintes ont capturé un bug réel en cours de développement : un mapping de valeurs dans l'ETL produisait 'Other' (majuscule) au lieu de 'other', rejeté par PostgreSQL (CheckViolation) mais indétecté par les tests unitaires du module, qui mockaient la base — cf. §8.

- **Cohérence inter-colonnes** — CHECK portant sur plusieurs colonnes de la même ligne : `ended_at >= started_at` (`workout_sessions`, `subscriptions`), ou la cohérence B2B (`tier = 'b2b' AND organization_id IS NOT NULL`) OR (`tier <> 'b2b' AND organization_id IS NULL`) qui garantit qu'une organisation n'est renseignée que pour les abonnements B2B, et inversement.
- **Idempotence au chargement** — `food_items` et `exercises` portent `UNIQUE(source, external_id)` : la paire (origine de la donnée, identifiant dans la source) sert de clé d'upsert (`INSERT ... ON CONFLICT DO UPDATE`) pour que les DAG Airflow puissent être rejoués sans dupliquer les données. Point de vigilance documenté et transmis à l'équipe ETL : `biometric_measurements` porte `UNIQUE(user_id, measured_at)` et non `UNIQUE(user_id, source)` — si le chargement fixe `measured_at = now()` au moment de l'ingestion, chaque ré-exécution génère un nouveau relevé au lieu de mettre à jour l'existant.
- **Génération de données manquantes, sans casser l'idempotence** — les datasets sources n'ont pas de champ nom (`first_name/last_name`, NOT NULL en base). Convention retenue : `Faker` avec une seed déterministe par ligne source (`Faker.seed(hash(patient_id))`), pas un tirage aléatoire à chaque exécution — sinon un même utilisateur changerait de nom à chaque ré-exécution du DAG, ce qui casse la logique d'idempotence par ailleurs garantie par les `UNIQUE(source, external_id)`.

4 Pipeline ETL

4.1 Structure du pipeline

Le pipeline suit un découpage classique en trois étapes, une responsabilité par module sous `etl/` :

- `extract/extract.py` — télécharge les quatre jeux de données Kaggle (nutrition, diet, gym-members, fitness-tracker) et interroge l'API publique `oss.exercisedb.dev` (pagination gérée, tolère un 429 en cours de pagination sans faire échouer tout le run) ; écrit les fichiers bruts sous `etl/data/`.
- `transform/` — un module par source (`clean_nutrition.py`, `clean_exercises.py`, `clean_diet.py`, `clean_users.py`), chacun responsable du renommage des colonnes vers le schéma cible, du typage, et du remplissage des colonnes obligatoires absentes de la source (cf. §3.2, convention `Faker.seed()`).
- `load/postgres_loader.py` — chargement idempotent, deux techniques selon la table : `food_items/exercises` passent par une table de staging (`to_sql`) puis un `INSERT ... SELECT ... ON CONFLICT DO UPDATE` depuis le staging (clé `UNIQUE(source, external_id)`) ; `users` par un `INSERT ... ON CONFLICT (email) DO UPDATE ... RETURNING user_id` direct, dont l'identifiant sert ensuite à insérer les tables dépendantes (`biometric_measurements`, `workout_sessions`, `medical_profiles`, `dietary_preferences`).

`extract` et `load` exposent chacun une fonction d'entrée (`run()`, `load_data()`) plutôt que d'exécuter leur logique au chargement du module — nécessaire pour un usage sûr depuis Airflow, qui importe périodiquement les fichiers DAG pour les parser sans vouloir déclencher un téléchargement ou une connexion base à chaque fois.

L'orchestration (Sprint 3) est un DAG Airflow unique, `healthai_etl_pipeline` (`extract` → `transform_and_load` → `quality_check`), plutôt qu'un DAG par source comme envisagé initialement : le volume et le nombre de sources de ce projet ne justifient pas la complexité d'une orchestration plus fine. `transform` et `load` sont une seule tâche parce qu'ils s'exécutent déjà dans le même processus côté code, sans artefact intermédiaire sur disque — les séparer aurait forcé à faire transiter des DataFrames pandas par XCom, déconseillé par la documentation Airflow au-delà de très petits volumes. `quality_check` exécute des règles réelles (tables vides, valeurs plausibles mais suspectes, complétude) et journalise dans `data_quality_log` — jusqu'ici rien n'écrivait de vraies anomalies dans cette table en dehors du seed de démonstration.

Isolation de dépendances. Airflow 2.10.5 exige `SQLAlchemy<2.0` en interne, incompatible avec le 2.0.51 utilisé partout ailleurs dans le projet (installer les deux dans le même environnement Python casse le webserver/scheduler — confirmé par les avertissements `pip` avant correction). Les dépendances ETL sont donc installées dans un venv dédié à l'intérieur de l'image Airflow (`/opt/etl-venv`), et chaque

tâche du DAG est un **BashOperator** qui invoque ce venv en sous-processus plutôt qu'un **PythonOperator** qui importerait `etl/` dans le process Airflow lui-même.

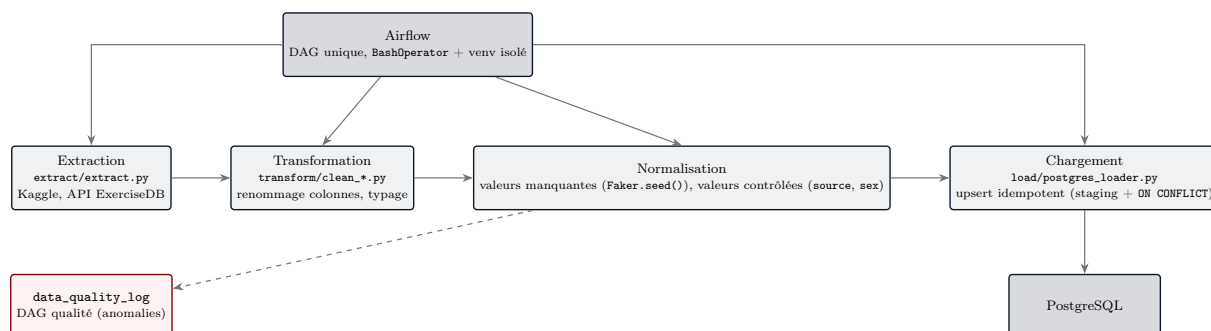


FIGURE 4 – Flux ETL : extraction (téléchargement brut vers `etl/data/`), transformation (renommage/typage), normalisation (remplissage déterministe des colonnes obligatoires absentes de la source, contrôle des valeurs), chargement (upsert idempotent) — orchestré par le DAG Airflow unique `healthai_etl_pipeline`, avec journalisation des anomalies réelles dans `data_quality_log`.

4.2 Données sources

Source	Alimente	Valeur source
Kaggle — Daily Food and Nutrition	<code>food_items</code>	<code>kaggle_daily_food_nutrition</code>
Kaggle — Diet Recommendations	<code>medical_profiles</code> , <code>dietary_preferences</code>	(pas de source)
Kaggle — Gym Members Exercise	<code>users</code> , <code>workout_sessions</code>	<code>kaggle_gym_members</code>
Kaggle — Fitness Tracker	<code>users</code> , <code>workout_sessions</code>	<code>kaggle_gym_members</code>
API <code>oss.exercisedb.dev</code>	<code>exercises</code>	<code>exercisedb</code>

TABLE 2 – Correspondance source → table(s) alimentée(s). La colonne `source` est la clé de traçabilité et, combinée à `external_id`, la clé d'upsert idempotent pour `food_items/exercises` (cf. §3.2).

Point de vigilance transverse, documenté dans `docs/plan_de_developpement.md` et vérifié par relecture croisée avant fusion des PR de l'équipe ETL : la valeur exacte de `source` doit correspondre caractère par caractère à celle attendue par la clé d'upsert et par `database/seed/seed_data.py` (référence de mapping colonnes → tables) — une divergence silencieuse (ex. `'kaggle_food_nutrition'` au lieu de `'kaggle_daily_food_nutrition'`) romprait la déduplication sans lever d'erreur SQL, puisque `source` n'est contrainte par aucun `CHECK` de valeurs. Ce type d'erreur ne se détecte pas par les tests unitaires du module `transform` (entrées mockées, aucun accès à une vraie base) : seule une vérification contre le schéma réel ou contre `seed_data.py` le révèle.

4.3 Qualité des données

`data_quality_log` centralise les anomalies détectées par l'ETL (Sprint 3, tâche `quality_check` du DAG `healthai_etl_pipeline`, `etl/quality/checks.py`) : table source, ligne concernée le cas échéant, règle déclenchée, sévérité (`info/warning/error/critical`), message. Consultation, filtrage et résolution manuelle des anomalies via l'interface admin (Sprint 5, §6). La contrainte `chk_data_quality_log_resolution_consistency` garantit qu'une anomalie non résolue ne porte ni `resolved_at` ni `resolved_by` — cohérence vérifiée par test (`tests/data_quality/test_schema.py`).

5 API REST

5.1 Structure

FastAPI (`api/main.py`), toutes les routes préfixées `/api/v1`. Structure en couches, une responsabilité par dossier :

- **routers/** — la couche HTTP : un routeur par ressource, déclare les endpoints, valide les entrées via les schémas Pydantic, délègue à **crud/**.
- **crud/** — accès base de données (requêtes SQLAlchemy), indépendant de FastAPI, ce qui permet de le tester sans monter d'application HTTP.
- **models/** — les entités SQLAlchemy (ORM), une classe par table.
- **schemas/** — les modèles Pydantic de requête/réponse (`...Create`, `...Read`, `...Update`), séparés des **models/** pour ne jamais exposer directement une colonne sensible (ex. `hashed_password`) par accident.

Authentification par JWT (OAuth2 password flow) : `POST /auth/login` renvoie un jeton, envoyé ensuite en `Authorization: Bearer <token>`. Trois dépendances FastAPI réutilisables (`api/core/deps.py`) portent le contrôle d'accès : `CurrentUser` (authentifié), `CurrentAdmin` (403 si `is_admin` faux), `CurrentPremiumUser` (403 hors palier payant, cf. §5.3) — déclarées une fois, réutilisées comme simple type d'argument dans la signature de chaque route protégée. Documentation interactive auto-générée : `/docs` (Swagger UI), `/redoc`.

5.2 Endpoints principaux

- **auth** — inscription, connexion (émission du JWT).
- **users / admin** — profil de l'utilisateur authentifié ; gestion de tout compte, réservée aux admins.
- **food-items / exercises** — catalogues alimentés par l'ETL, lecture libre, écriture admin uniquement.
- **nutrition-logs / workouts / biometrics / health-profiles** — données propres à l'utilisateur authentifié (repas journalisés ; séances et séries d'entraînement ; relevés biométriques ; profils médical/alimentaire/de forme historisés et recommandations diététiques en lecture seule).
- **data-quality** — vue admin sur `data_quality_log` (cf. §4.3) et résolution des anomalies, alternative programmatique à l'interface Gradio.
- **subscriptions / organizations** — auto-gestion de l'abonnement ; provisionnement B2B et gestion des organisations partenaires, admin uniquement (cf. §5.3).
- **ai** — génération de contenu (recommandations, plans), réservée aux paliers payants (cf. §5.3).
- **health** — *liveness check* pour la supervision.

5.3 Abonnements et microservices IA

Quatre paliers (`subscriptions.tier`) : `free` (par défaut à l'inscription), `premium`, `premium_plus`, et `b2b` (rattaché à une `organization`, provisionné par un admin — pas de self-service pour ce palier). Une seule souscription `active` à la fois par utilisateur (index unique partiel), changement de palier = clôture de l'ancienne + ouverture d'une nouvelle.

Le contenu IA (`POST /ai/diet-recommendations`, `/ai/workout-plans`, `/ai/nutrition-plans`) est réservé aux paliers payants via une dépendance dédiée (`CurrentPremiumUser`, `api/core/deps.py`) : 403 pour un compte `free`, 401 sans authentification. Le microservice IA lui-même n'est pas encore déployé (cf. issue #49) : les routes et le stockage (tables `workout_plans`, `nutrition_plans`, plus le `diet_recommendations` existant) sont préparés à l'avance, avec un client stub (`api/services/ai_client.py`) reproduisant la signature du futur appel HTTP réel. Objectif : brancher le vrai microservice n'impliquera de changer que ce fichier, ni les routers ni le contrat `POST /ai/...` exposé aux clients. Vérifié par 136 tests réels contre PostgreSQL (gating par palier, persistance, isolation des données par utilisateur), cf. §8.

6 Interface admin (Gradio) et dashboard Metabase

6.1 Interface admin

Outil interne (Sprint 5, `admin_interface/app_gradio.py`, thème Gradio Soft) destiné aux administrateurs (`users.is_admin`) pour traiter les anomalies journalisées dans `data_quality_log` sans accès direct à la base :

- **Consultation** — tableau filtrable par sévérité, statut de résolution et table source (filtre partiel), rafraîchi à la demande.
- **Correction manuelle** — résolution d’une anomalie par son `dq_log_id`, avec traçabilité optionnelle de l’administrateur ayant traité le cas (`resolved_by`).
- **Export** — CSV et JSON du résultat filtré courant, dates sérialisées au format ISO.

Testé par 26 tests (`tests/admin_interface/`) : logique pure (formatage, traduction des filtres) sans dépendance, et logique base de données sur un PostgreSQL jetable (cf. §8).

Qualité des données

Consultation, correction manuelle et export des anomalies détectées par l'ETL

Filtres

Sévérité	Statut de résolution	Table source (filtre partiel)	Rafraîchir
Toutes	Non résolues	ex. food_items	

Anomalies détectées

dq_log_id	source_table	source_record_id	dag_id	rule_name	severity	message	detected_at	reso
1	food_items			check_negative_values	warning	Valeur négative détectée sur une co...	2026-07-09 12:11	fals
2	nutrition_logs			check_fk_integrity	error	food_item_id introuvable dans food_...	2026-07-09 12:11	fals
3	biometric_measure...			check_range	info	body_fat_pct hors de la plage 0-100...	2026-07-09 12:11	fals

Correction manuelle

Identifiant de l'anomalie (dq_log_id)	Résolu par (administrateur)	Marquer comme résolu
Numéro affiché dans la colonne 'dq_log_id' du tableau ci-dessus.	Optionnel : sélectionnez le compte admin qui traite cette anomalie, pour tracer qui a résolu quoi. Laissez sur 'Non renseigné' si vous ne souhaitez pas l'indiquer.	
0	admin@healthai.local (#1)	

Export

Exporter en CSV	Exporter en JSON
Fichier exporté	

FIGURE 5 – Interface admin — filtres, consultation des anomalies, correction manuelle et export sur une même page.

6.2 Accessibilité RGAA

Un audit automatisé (`axe-core`, navigateur headless) a été mené sur l’interface plutôt qu’une simple relecture visuelle, pour objectiver le niveau de conformité RGAA/WCAG AA réellement atteint. Résultat, deux catégories distinctes :

Corrigé, propre au code applicatif —

- *Contraste insuffisant* : les couleurs de texte par défaut du thème Soft échouaient au seuil AA de 4,5 :1 — labels de champs à 4,34 :1, texte d’aide (`info=`) à 2,56 :1 (échec net). Cause : `block_label_text_color` et `block_info_text_color` valent respectivement `*neutral_500/*neutral_400` par défaut dans Gradio. Corrigé en les fixant explicitement à `*neutral_600 (#475569)`, ce qui porte le ratio à 6,1–7,6 :1 selon le fond concerné (vérifié par calcul de luminance relative sur les trois fonds rencontrés : blanc, `#f1f5f9`, `#e2e8f0`).

— *Hiérarchie de titres invalide* : le titre principal (h1) était suivi directement de sous-titres en h3 (Filtres, Correction manuelle, Export), sans niveau h2 intermédiaire. Corrigé en repassant ces sous-titres en h2.

Non corrigible depuis le code applicatif — le composant `Dataframe` de Gradio génère, pour son mécanisme de glisser-déposer CSV/TSV, un bouton englobant l'ensemble des cellules et en-têtes du tableau (chacun rendu comme `span role="button"` individuellement focusable), ce qui viole plusieurs règles ARIA : éléments interactifs imbriqués (`nested-interactive`), noms accessibles manquants sur les cellules (`aria-command-name`), structure de grille invalide (`aria-required-children`). Confirmé indépendamment par un test de navigation clavier (parcours Tab sur 8 arrêts) : le tableau se comporte comme un unique arrêt de focus englobant, sans granularité cellule par cellule utilisable au clavier/lecteur d'écran. Ces défauts sont internes au rendu du composant Gradio, documentés comme limitation connue plutôt que contournés (un contournement aurait nécessité de reconstruire le composant tableau, hors de portée du calendrier).

6.3 Dashboard Metabase

Instance self-hosted (`docker compose`, cf. §7), connectée en lecture à PostgreSQL. Au-delà des 14 tables du schéma, plusieurs vues SQL (`vw_biometric_latest_per_user`, `vw_nutrition_calories_by_category`, `vw_workout_sessions_weekly`...) sont exposées pour servir de base à des KPI (cf. figure 6), en complément de l'interface admin — Metabase pour la vue d'ensemble/tendances, l'interface admin pour l'action (résolution au cas par cas des anomalies, figure 7).

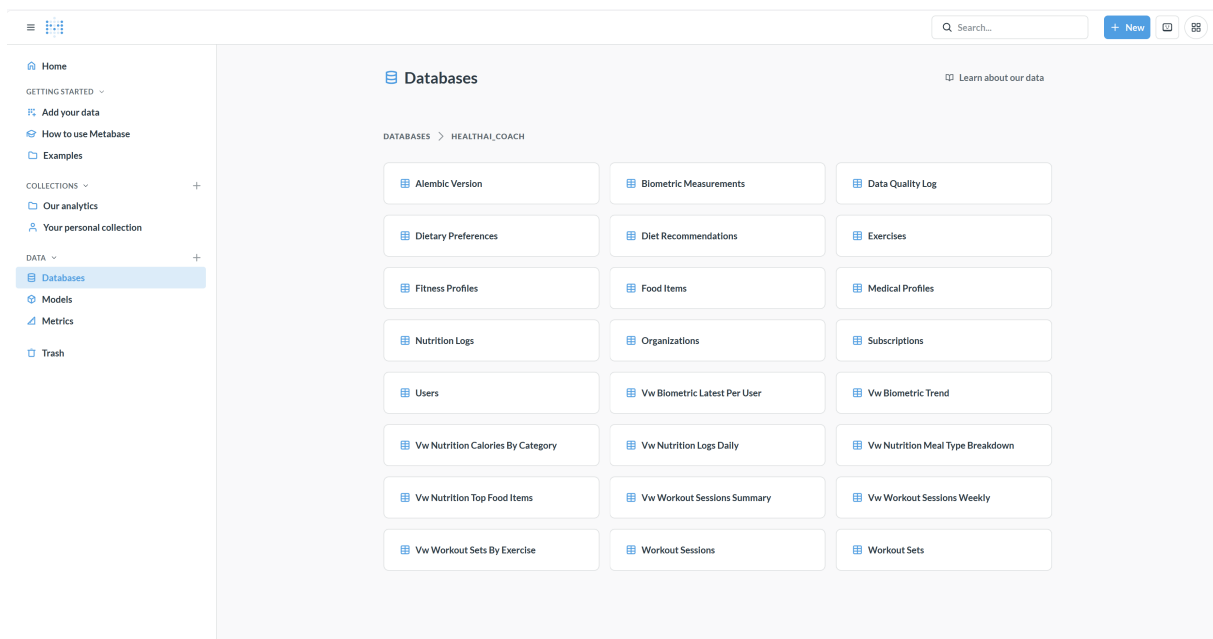


FIGURE 6 – Metabase — schéma `healthai_coach` exposé : les 14 tables et les vues KPI pré-construites.

Dq Log ID	Source Table	Source Record ID	Diag ID	Rule Name	Severity	Message	Detected At	Resolved	Resolved At	Resolved By
1	food_items			check_negative_values	warning	Valeur négative détectée sur une colonne nutritionnelle avant nettoyage.	July 9, 2026, 12:11 PM	false		
2	nutrition_logs			check_fk_integrity	error	food_item_id introuvable dans food_items au moment de l'ingestion.	July 9, 2026, 12:11 PM	false		
3	biometric_measurements			check_range	info	body_fat_pct hors de la plage 0-100, ligne écartée.	July 9, 2026, 12:11 PM	false		

FIGURE 7 – Metabase — consultation de `data_quality_log`, données réellement chargées en base.

Limitation assumée, propre au plan gratuit self-hosted : la personnalisation visuelle est restreinte à **Admin > Paramètres > Apparence** (nom de l'application, favicon, couleur primaire); le retrait du branding « Powered by Metabase », les palettes de couleurs de graphiques et les polices personnalisées sont réservés à l'offre Enterprise. Aucune méthode open source équivalente n'a été identifiée pour contourner

cette restriction sans re-packager Metabase soi-même (hors périmètre). Décision : conserver le rendu par défaut plutôt que d'investir du temps sur ce point avant l'échéance.

7 Dockerisation

Objectif Sprint 6 : `docker compose up` → environnement fonctionnel en moins de 30 minutes, une seule commande. Sept services (`docker-compose.yml`) :

- **postgres** (image officielle 16) — healthcheck `pg_isready`, condition de démarrage pour les services suivants. Au tout premier démarrage (volume vierge), `database/init/*.sql` crée aussi les bases `metabase` et `airflow`, séparées de `healthai_coach`.
- **db-init** — conteneur à usage unique (`service_completed_successfully`) : applique les migrations Alembic puis peuple la base via `database/seed/seed_data.py` (données fictives, dev/démo).
- **admin_interface** — démarre une fois `db-init` terminé avec succès, expose le port 7860.
- **metabase** — image officielle, même dépendance à `db-init`, expose le port 3000 ; connexion à PostgreSQL configurée une seule fois manuellement au premier lancement (Metabase n'expose pas d'API de pré-configuration).
- **airflow-init** — conteneur à usage unique : migre la base de métadonnées Airflow et crée le compte admin, via l'entrypoint officiel de l'image (`_AIRFLOW_DB_UPGRADE`, `_AIRFLOW_WWW_USER_CREATE`).
- **airflow-webserver** / **airflow-scheduler** — démarrent une fois `airflow-init` terminé, exposent l'UI sur le port 8080. `LocalExecutor` + base `airflow` partagée sur le postgres du projet plutôt que le stack `CeleryExecutor`+Redis complet vu en formation (config commune factorisée via `x-airflow-common`, même principe que le patron officiel Airflow) : un DAG unique ne justifie pas plusieurs workers, et la machine de démo n'a pas la RAM pour 7 services Airflow en plus du reste de la stack applicative.

`api` reste en stub commenté dans le compose au moment de la rédaction, en attente du `Dockerfile` de l'équipe API (Rôle B) — les variables d'environnement sont déjà prévues dans `.env.example`.

Point de vigilance — `db-init` ne deviendra pas le vrai pipeline ETL. `db-init` exécute `seed_data.py`, qui commence par un `TRUNCATE ... CASCADE` avant repeuplement : adapté au dev local et à la démo, mais incompatible avec le vrai pipeline (`etl/load`, Sprint 2), conçu pour upsert sans jamais tronquer et tourner de façon récurrente via le DAG Airflow `healthai_etl_pipeline` (Sprint 3). **Confirmé empiriquement, pas seulement théorique** : en testant le DAG (extraction réelle Kaggle/ExerciseDB, 2851 users, 645 food_items, 404 exercices chargés), un simple redémarrage d'`admin_interface` a redéclenché `db-init` qui a immédiatement tout effacé pour revenir aux 101/50/4 lignes de démo. Décision toujours non tranchée à date de rédaction, deux options documentées (`docs/guide_deploiement.md`) : retirer `seed_data.py` de `db-init` et ne garder que les migrations, ou conditionner son exécution à une variable d'environnement dédiée réservée aux environnements de démo.

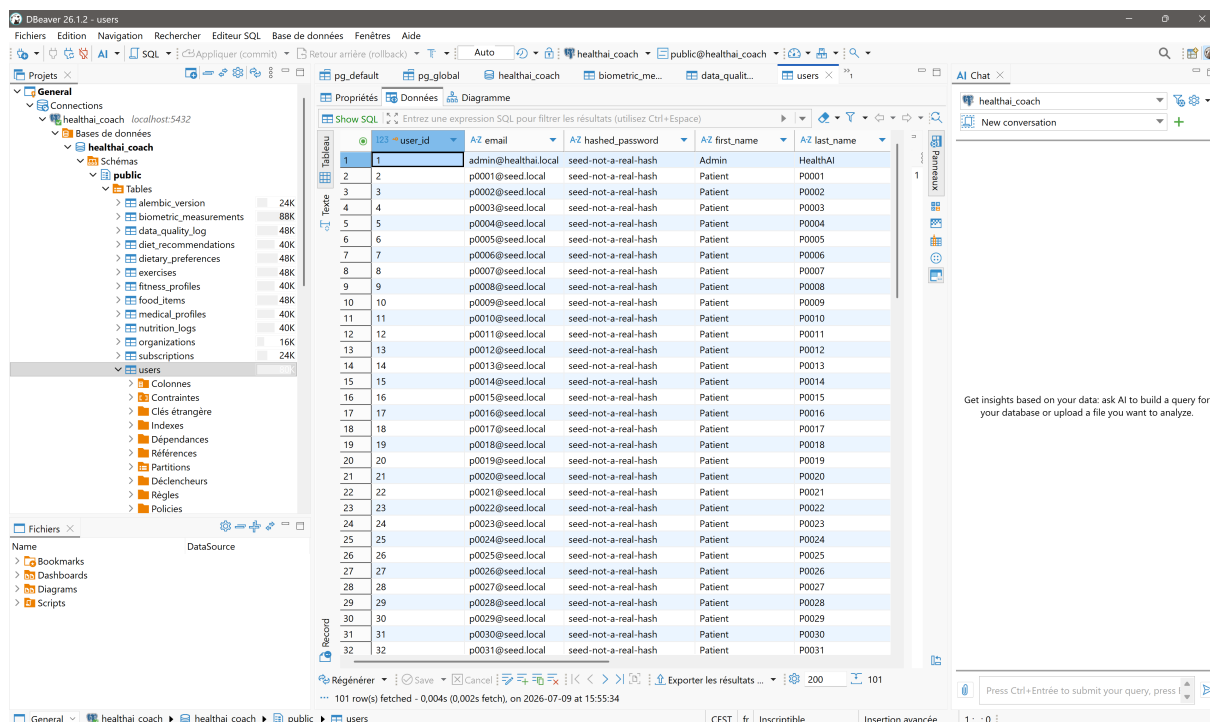


FIGURE 8 – Vérification du chargement réel des données (**users**, 101 lignes) via DBeaver connecté au conteneur postgres. On reconnaît la convention de seed dev/démo (email en `pXXXX@seed.local`, `hashed_password = 'seed-not-a-real-hash'`) — à distinguer des vraies données produites par l'ETL une fois load en production (cf. point de vigilance ci-dessus).

8 Tests et CI/CD

8.1 Suite de tests

Module	Périmètre	Fichiers
tests/data_quality/	Schéma SQL : tables, FK, CHECK, cascades	1
tests/admin_interface/	Interface admin : logique pure + DB réelle	3
tests/api/	Auth, users, organizations, subscriptions, contraintes	7
tests/etl/	Fonctions <code>transform</code> (mockées) + <code>quality/checks.py</code> (DB réelle)	3

TABLE 3 – Suite de tests (141 tests au moment de la rédaction), `pytest`.

Les tests `data_quality`, `admin_interface` et `etl/test_quality_checks.py` s'exécutent contre un PostgreSQL jetable (schéma reconstruit à chaque run depuis `ddl_postgres.sql`) plutôt que des mocks, précisément parce que les contraintes `CHECK/FOREIGN KEY` sont une partie significative des règles métier du projet (cf. §3.2) : un mock ne les vérifie pas. Cette différence de méthode s'est révélée concrète en cours de développement — une valeur `'Other'` (majuscule) produite par le module `transform` de l'ETL passait les 4 tests unitaires du module (entrées mockées) mais violait la contrainte `CHECK` réelle en base, détecté uniquement par insertion effective lors de la relecture de la PR correspondante.

8.2 Pipeline CI

GitHub Actions (`.github/workflows/ci.yml`), déclenché sur `pull_request` et `push` vers `main/dev` : service PostgreSQL 16 éphémère, application des migrations Alembic (`alembic upgrade head`), puis `pytest tests/ -v`. Dépendances installées depuis deux fichiers consolidés (`database/migrations/requirements.txt` + `tests/requirements.txt`) plutôt que les `requirements.txt` propres à chaque module — point de vigilance pour la reproduction locale d'un échec CI : tester avec l'ensemble des `requirements.txt` du dépôt installés masque de vraies dépendances manquantes en CI (rencontré une fois en pratique, cf. §8).

9 Bilan et obstacles

9.1 Obstacles techniques rencontrés

- **Les tests mockés ne remplacent pas une vraie base** — une valeur 'Other' produite par l'ETL passait 4/4 tests unitaires (entrées mockées) mais violait une contrainte `CHECK` réelle ; seule une insertion contre un PostgreSQL jetable l'a révélée (cf. §3.2, §8). Décision prise en conséquence : les modules touchant au schéma sont désormais systématiquement re-testés contre une vraie base avant fusion, pas seulement via les tests unitaires du module.
- **Ma propre vérification manuelle de contraste RGAA était incomplète** — j'avais initialement validé le thème Gradio « à l'œil » sur quelques couleurs, en manquant les couleurs de texte `info=`/labels réellement utilisées. Un audit automatisé (`axe-core`) a révélé l'erreur (2,56 :1 au lieu de 4,5 :1 requis) — corrigé depuis (cf. §6). À retenir : l'automatisation prime sur la relecture manuelle pour ce type de vérification.
- **Reproduction fidèle de la CI** — un premier échec CI (`ModuleNotFoundError: gradio`) n'était pas reproductible en local parce que l'environnement de test local avait plus de dépendances installées que la CI réelle (tous les `requirements.txt` du dépôt, alors que la CI n'en installe que deux). Depuis, toute validation de correctif reproduit exactement la séquence de commandes CI dans un environnement vierge avant d'être considérée acquise.
- **db-init vs pipeline ETL réel, confirmé en conditions réelles** — le risque théorique identifié dès la PR #44 (`TRUNCATE` de `db-init` vs `upsert` jamais destructeur du vrai pipeline) s'est concrètement produit en testant le DAG Airflow : un redémarrage anodin d'`admin_interface` a effacé des données réelles fraîchement chargées (cf. §7). La bascule (retirer `seed_data.py` de `db-init` ou la conditionner) reste à trancher, mais n'est plus un risque abstrait pour la suite du projet.
- **Conflit de version SQLAlchemy avec Airflow** — Airflow 2.10.5 exige `SQLAlchemy<2.0` en interne, alors que le reste du projet (y compris `etl/`) utilise 2.0.51 depuis le début. Installer les dépendances ETL dans le même environnement Python que Airflow les aurait fait cohabiter et aurait risqué de casser le webserver/scheduler — détecté via les avertissements `pip` au moment du build de l'image, avant tout déploiement. Résolu par un venv Python isolé à l'intérieur de l'image Airflow, les tâches du DAG l'invoquant en sous-processus plutôt que d'importer `etl/` dans le process Airflow.

9.2 Points positifs

- Contraintes d'intégrité (`CHECK`, `FOREIGN KEY`, index uniques partiels) portées directement par PostgreSQL plutôt que dans le code applicatif : les règles métier survivent à n'importe quel point d'entrée (API, ETL, script ad hoc), pas seulement à un chemin de code particulier.
- Schéma auto-documenté (`COMMENT ON TABLE/COLUMN`) : les conventions non triviales (clé d'`upsert`, distinction cholestérol alimentaire/sanguin, etc.) sont lisibles depuis la base elle-même, pas seulement dans un document externe qui peut diverger du schéma réel.
- Convention d'idempotence (`UNIQUE(source, external_id)`, `Faker.seed()` déterministe) définie une fois puis reprise à l'identique par l'équipe ETL sur chaque nouvelle source, et par l'équipe API pour le pattern d'historisation (`diet_recommendations` → `workout_plans/nutrition_plans`, cf. PR #52).
- Discipline de revue systématique (checkout isolé, tests rejoués contre PostgreSQL frais, jamais de confiance aveugle dans un diff ou un « les tests passent ») qui a détecté plusieurs bugs réels avant fusion plutôt qu'en production.

10 Conclusion

Le backend HealthAI Coach couvre, à date de rédaction, un schéma PostgreSQL de 16 tables avec ses migrations Alembic, une API REST authentifiée (utilisateurs, journaux nutrition/activité, abonnements, routes IA prêtes pour un microservice pas encore déployé), un pipeline ETL complet et orchestré (extraction, transformation, chargement idempotent et contrôle qualité enchaînés par un DAG Airflow testé de bout en bout contre des données réelles), une interface d'administration de la qualité des données audité RGAA, un dashboard Metabase, et une conteneurisation `docker compose` (sept services) permettant de démarrer l'ensemble — hors `api`, en attente de son `Dockerfile` — en une seule commande.

Le principal facteur limitant a été le temps disponible dans le cadre de cette formation, pas une difficulté technique bloquante : le microservice IA n'est qu'esquissé (stub côté API), l'incompatibilité

entre `db-init` et le vrai pipeline ETL est identifiée et documentée mais pas encore tranchée, et certaines limitations d’accessibilité du composant `Dataframe` de Gradio ne sont pas corrigibles sans reconstruire le composant lui-même. Le choix a été fait de prioriser la robustesse de ce qui est livré (contraintes en base, tests contre une vraie base de données, CI reproductible, pipeline ETL vérifié en conditions réelles plutôt que supposé fonctionnel) plutôt que de couvrir l’intégralité du périmètre imaginé au démarrage — un compromis assumé plutôt qu’un oubli.

A Extrait du schéma SQL — tables principales

Extrait de `database/schema/ddl_postgres.sql` illustrant les patterns récurrents du schéma : contraintes `CHECK` sur domaine de valeurs fermé, clé d’idempotence `UNIQUE`, contrainte de cohérence multi-colonnes, et FK optionnelle avec `ON DELETE SET NULL`.

```

1 CREATE TABLE users (
2     user_id          SERIAL PRIMARY KEY,
3     email            VARCHAR(255) NOT NULL UNIQUE,
4     hashed_password  VARCHAR(255) NOT NULL,
5     first_name       VARCHAR(100) NOT NULL,
6     last_name        VARCHAR(100) NOT NULL,
7     date_of_birth    DATE,
8     sex              VARCHAR(10) CHECK (sex IN ('F', 'M', 'other')),
9     is_admin         BOOLEAN NOT NULL DEFAULT FALSE,
10    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
11    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
12 );
13
14 CREATE TABLE food_items (
15     food_item_id     SERIAL PRIMARY KEY,
16     external_id      VARCHAR(100),
17     source            VARCHAR(50) NOT NULL,
18     name              VARCHAR(255) NOT NULL,
19     calories_kcal     NUMERIC(7,2) NOT NULL CHECK (calories_kcal >= 0),
20     ingested_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
21     UNIQUE (source, external_id) -- cle d'upsert idempotent (cf. S3.2)
22 );
23
24 CREATE TABLE data_quality_log (
25     dq_log_id        BIGSERIAL PRIMARY KEY,
26     source_table     VARCHAR(100) NOT NULL,
27     rule_name        VARCHAR(150) NOT NULL,
28     severity         VARCHAR(20) NOT NULL
29     CHECK (severity IN ('info', 'warning', 'error', 'critical')),
30     message          TEXT NOT NULL,
31     detected_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
32     resolved         BOOLEAN NOT NULL DEFAULT FALSE,
33     resolved_at      TIMESTAMPTZ,
34     resolved_by      INTEGER,
35     CONSTRAINT fk_data_quality_log_resolved_by
36         FOREIGN KEY (resolved_by) REFERENCES users(user_id) ON DELETE SET NULL,
37     CONSTRAINT chk_data_quality_log_resolution_consistency
38         CHECK (resolved = TRUE OR (resolved_at IS NULL AND resolved_by IS NULL))
39 );

```